

Crypto Analytics Web Application

Техничка документација

Изработено од:

Бојана Брашнарска, 231108

Јована Андова, 231080

Иван Марковски, 231019

1. Преглед на системот

Оваа документација го опишува целиот систем за **Crypto Analytics Web Application**, кој се состои од:

- **Backend (Spring Boot REST API + H2 database)**
- **Frontend (React web app)**

Системот обезбедува:

- листа на crypto assets (coins),
- historical OHLCV податоци (Open/High/Low/Close/Volume),
- Technical Analysis (TA) сигнали и score за 1D/1W/1M.

Frontend апликацијата комуницира со backend преку HTTP requests кон REST API.

2. Технологии и алатки

Backend

- Java (Spring Boot)
- Spring Web (REST Controllers)
- Spring Data JPA (Hibernate ORM)
- H2 Database (file-based)
- Apache Commons CSV (CSV import)

Frontend

- React (Create React App)
- react-router-dom (routing)
- recharts (charts/graphs)
- @testing-library/* (testing utilities)

3. Архитектура на системот

3.1 Архитектура на високо ниво

React Frontend (<http://localhost:3000>)

|
| [fetch\(\)](#) / HTTP calls



Spring Boot Backend (<http://localhost:8080>)

|



H2 Database (file: [./data/demo-db](#))

3.2 Словевита архитектура (Backend)

- Controller layer (REST endpoints)
- Service layer (business logic)
- Repository layer (database access)
- Entity layer (data model)

4. Документација за Backend

4.1 Конфигурација на апликацијата (H2)

`application.properties`:

```
spring.datasource.url=jdbc:h2:file:./data/demo-db;AUTO_SERVER=TRUE
```

```
spring.datasource.driverClassName=org.h2.Driver
```

`spring.datasource.username=sa`

`spring.datasource.password=`

`spring.jpa.hibernate.ddl-auto=update`

`spring.jpa.show-sql=false`

`spring.h2.console.enabled=true`

`spring.h2.console.path=/h2`

H2 Console:

`http://localhost:8080/h2`

- JDBC URL: `jdbc:h2:file:./data/demo-db`
- username: `sa`
- password: (empty)

4.2 CORS конфигурација

Backend овозможува CORS за да може frontend (React) да пристапува до API.

CorsConfig дозволува:

- Allowed origin: `http://localhost:3000`
- Methods: `GET, POST, PUT, DELETE, OPTIONS`
- Headers: `*`

Забелешка: Некои контролери користат и `@CrossOrigin(origins="*")` (OK за dev).

4.3 Модел на податоци (Entities)

4.3.1 CryptoAsset (табела: `crypto_asset`)

Field	Type	Description
id	Long	Primary key
coinId	String	Unique identifier (пример: <code>bitcoin</code>)
symbol	String	Symbol (пример: <code>btc</code>)
name	String	Name
rank	Integer	Market cap rank
currentPrice	BigDecimal	Current price

4.3.2 CoinHistory (табела: `coin_history`)

Field	Type	Description
id	Long	Primary key
asset	CryptoAsset	Many-to-One (FK)
openTime	Instant	Candle time

open	Double	Open price
high	Double	High price
low	Double	Low price
close	Double	Close price
volume	Double	Volume

Constraints / Indexes:

- Unique index: (asset_id, open_time) → нема дуплікати

4.3.3 TaResult (табела: ta_result)

Field	Type	Description
id	Long	Primary key
coinId	String	Unique coin id
symbol	String	Symbol (BTC)
name	String	Name
signal1D	String	BUY/SELL/HOLD

score1D	double	Score for 1D
signal1W	String	BUY/SELL/HOLD
score1W	double	Score for 1W
signal1M	String	BUY/SELL/HOLD
score1M	double	Score for 1M
updatedAt	LocalDateTime	Last update timestamp

4.4 Иницијализација на податоци (CSV import)

4.4.1 DataLoader (assets + history)

При стартување (@PostConstruct) се прави import **само ако табелите се празни**:

- `top_1000_coins.csv` → `crypto_asset` (до 1000 записи)
- `coin_details.csv` → `coin_history` (OHLCV податоци)

Карактеристики:

- batch insert (10,000 записи по `saveAll`)
- deduplication логика пред insert (за да не се внесуваат дупликати)

4.4.2 TA import при стартување

`TaStartupRunner` автоматски импортира:

- `data/TA_top100_summary.csv` → `ta_result`

`TaImportService` прави upsert:

- ако постои `coinId` → update
- ако не → insert

4.5 REST API референца (Backend)

Base URL: `http://localhost:8080`

4.5.1 Crypto Assets API

Base path: `/api`

Method	Endpoint	Description
GET	<code>/api/all</code>	Get all assets
GET	<code>/api/{id}</code>	Get asset by DB id
POST	<code>/api/add</code>	Create asset
PUT	<code>/api/update/{id}</code>	Update asset
DELETE	<code>/api/delete/{id}</code>	Delete asset

4.5.2 Coin History API

Base path: `/api/history`

Method	Endpoint	Params	Description
GET	/api/history/symbol/{symbol}	optional days	Last 500 candles by symbol
GET	/api/history/symbol/{symbol}?days=30	days=30	Filter to last N days

Response example:

```
{
  "openTime": "2025-12-01T00:00:00Z",
  "open": 100.0,
  "high": 110.0,
  "low": 95.0,
  "close": 105.0,
  "volume": 12345.0
}
```

4.5.3 TA API

Base path: /api/ta

Method	Endpoint	Description
GET	/api/ta	Get all TA results
GET	/api/ta/{coinId}	TA by coinId

GET	/api/ta/symbol/{symbol}	TA by symbol
POST	/api/ta/import	Manual CSV import

4.5.4 Utility endpoints (CryptoController)

Method	Endpoint	Description
GET	/api/counts	Counts: assets + history
GET	/api/assets	All assets (utility)
GET	/api/history/{coinId}	Last 500 by coinId

5. Документација за Frontend (React)

5.1 Структура на проектот

Frontend проектот е React app со следна структура (пример од [src/](#)):

components/

- [Layout.js](#) (главен layout: sidebar + top bar + content)
- [Sidebar.js](#) (navigation)
- [DashboardKpiBar.js](#) (KPI cards)
- [SignalBadge.js](#) (визуелен badge BUY/SELL/HOLD)
- [TaGauge.js](#) (TA gauge визуелизација)

- `TaCoinsTable.js` (табела за coins/TA)
- `TaSummaryCard.js` (summary card за TA)

pages/

- `Home.js` (почетна страна)
- `Dashboard.js` (dashboard view)
- `CoinsPage.js` (листинг на coins)
- `CoinDetailsPage.js` (детали за coin + графици)
- `Settings.js`, `Reports.js`, `Profile.js`, `List.js` (останати страници)

Останати важни фајлови:

- `api.js` (API_BASE конфигурација)
- `App.js` (routes)
- `index.js` (React root render)
- `App.css`, `index.css` (styling)
- `package.json` (dependencies + scripts)

5.2 Конфигурација за повик кон backend (API)

Во frontend се користи:

```
export const API_BASE = "http://localhost:8080";
```

Сите API calls треба да одат кон `API_BASE`, пример:

- `${API_BASE}/api/assets`

- `${API_BASE}/api/history/symbol/btc`
- `${API_BASE}/api/ta/symbol/BTC`

5.3 Руты и навигација (Routing)

Frontend користи `react-router-dom` и routes се дефинирани во `App.js`. Главни routes:

Route	Page	Description
<code>/</code>	redirect → <code>/dashboard</code>	Default redirect
<code>/dashboard</code>	Dashboard	KPI + overview
<code>/coins</code>	CoinsPage	Табела/листинг coins
<code>/coin/:symbol</code>	CoinDetailsPage	Детали по symbol
<code>/settings</code>	Settings	Settings page
<code>/reports</code>	Reports	Reports page
<code>/profile</code>	Profile	Profile page

`Layout` компонентата ги обвиткува страниците и обезбедува конзистентен UI (sidebar + topbar).

5.4 Кориснички интерфејс (Layout + Sidebar + Topbar)

Frontend има "dashboard" стил layout:

- fixed sidebar (лево)
- top bar со search поле и user badge
- main content area со page title, KPI cards и табели

Ова е имплементирано преку CSS класи како:

- `.app-shell`, `.sidebar`, `.main-area`, `.top-bar`, `.content`
 - `.kpi-row`, `.kpi-card`, `.crypto-table`
-

5.5 Styling (CSS)

Во проектот се користат два главни CSS фајла:

- `index.css` (global стилови + reset)
- `App.css` (layout, pages, dashboard UI)

Клучни стилови:

- dashboard layout со sidebar + topbar
- KPI cards во grid
- табела со `border-collapse` и zebra редови (alternating rows)

5.6 Dependencies и scripts (package.json)

Клучни dependencies:

- `react, react-dom`
- `react-router-dom`
- `recharts`
- `react-scripts`
- `web-vitals`
- `@testing-library/*`

Scripts:

- `npm start` (development)
- `npm build` (production build)
- `npm test` (tests)

6. Стартирање на системот (Backend + Frontend)

6.1 Стартирање на Backend

1. Отвори backend проект (Spring Boot)
2. Start application (Run од IDE или `mvn spring-boot:run`)
3. Backend работи на: `http://localhost:8080`
4. H2 Console: `http://localhost:8080/h2`

При прв старт:

- Hibernate креира/ажурира табели
- CSV import се извршува ако табелите се празни

6.2 Стартирање на Frontend

1. Отвори frontend folder
2. Install dependencies:

`npm install`

3. Start React app:

`npm start`

4. Frontend работи на: `http://localhost:3000`
-

7. Интеграција (Frontend ↔ Backend)

Frontend ги користи backend endpoints за:

- coins list (assets API)
- coin details + history chart (history API)
- TA сигнали и score (TA API)

CORS е овозможен во backend за да дозволи requests од React.

8. Заклучок

Crypto Analytics Web Application е full-stack систем кој комбинира:

- backend REST API со стабилен data layer и автоматски CSV import
- React frontend со routing, dashboard layout и визуелизации (табели/графици)

Архитектурата е модуларна и овозможува лесно проширување со нови аналитички функционалности.